DAA Final Exam Revision Pack

20MCA203 - Modules 1, 2 and the specified part of Module 5

Exam scope used: Module 1 in full; Module 2 in full; Module 5 only: Approximation Ratio, 2-approximation for Vertex Cover, and Vertex Cover Approximation using Linear Programming and LP Rounding. Randomized algorithms, Modules 3 and 4 are excluded.

Source-file check: Your Module 1 and Module 2 files cover the relevant syllabus topics. Several files uploaded under Module 5, however, contain randomized algorithms or Module 3 material such as backtracking, branch and bound, and lower-bound theory—not the specified Vertex Cover approximation content. To avoid mixing excluded topics into your revision, the Module 5 section below is based on your stated exam scope and its repeated appearance in the uploaded DAA question papers. I have not treated those unrelated pages as your approximation notes.

1. What the previous papers suggest

I compared the distinct DAA papers visible in the uploaded set: December 2021, December 2022, December/January 2023-24, May 2024 and December 2024. Some uploaded PDFs are duplicate scans or bundles, so they are not counted as additional sittings.

Module 1 repeated pattern

- **Asymptotic notations:** a 6-mark question in Dec 2021, Dec 2022 and May 2024.
- **Merge Sort:** 6-mark question in Dec 2021, Dec 2022 and Dec 2024.
- **Quick Sort:** algorithm/tracing in Dec 2023-24 and May 2024; its worst-case O(n^2) appeared as a short question in Dec 2024.
- **Recurrence equations:** solving by iteration/substitution appeared in Dec 2021, Dec 2023-24 and Dec 2024.
- **Divide and conquer/matrix multiplication:** short question in Dec 2022; Strassen's matrix multiplication in Dec 2024.

Module 2 repeated pattern

- **Greedy control abstraction:** short-answer question in Dec 2021, Dec 2022 and May 2024.
- **Minimum spanning tree:** Prim's or Kruskal's algorithm appeared in Dec 2021, Dec 2022 and Dec 2024.
- **Fractional knapsack:** a worked numerical question in Dec 2023-24 and May 2024.

- **Shortest paths / dynamic programming:** all-pairs shortest path appeared in Dec 2021 and Dec 2023-24; a single-source shortest-path question appeared in Dec 2022.
- **Travelling Salesperson Problem:** appeared in May 2024 and Dec 2024.
- **Job scheduling with deadlines** is in your syllabus but was not a standout repeat in the distinct papers reviewed. Still prepare the greedy steps.

Module 5 - only the allowed topics

- **2-approximation for Vertex Cover and its ratio proof:** appeared in Dec 2021, Dec 2022 and May 2024.
- **LP formulation/LP rounding for Vertex Cover:** appeared as a 6-mark question in Dec 2023-24.
- **Approximation ratio:** repeatedly appears in the short-answer section.

High-priority conclusion (not a guarantee): prepare the 6-mark answers for asymptotic notations, Merge Sort, Quick Sort/recurrences, Prim/Kruskal, fractional knapsack, Floyd-Warshall, TSP, and both Vertex Cover approximation methods first.

MODULE 1 - Algorithm Analysis and Divide & Conquer

2. Time and space complexity

Time complexity describes how the number of basic operations grows with input size n. It is not normally a measurement in seconds; it expresses growth as n increases.

Space complexity describes the memory required as input size grows. It can include input storage and auxiliary space used by the algorithm; state which convention you are using.

Cases: - Best case: minimum work for any input of size n. - Worst case: maximum work for any input of size n. - Average case: expected work over a stated distribution of inputs.

Examples: Linear Search has best-case O(1) when the first item matches, and worst-case O(n) when the item is last or absent. Merge Sort is Theta(n log n) in best, average and worst cases.

3. Asymptotic notations

Asymptotic notation describes growth for sufficiently large n while ignoring constant factors and lower-order terms.

- **Big O, O(g(n)): upper bound.** There are constants c > 0 and n0 such that 0 <= f(n) <= c g(n) for every n >= n0.
- **Big Omega, Omega(g(n)): lower bound.** There are constants c > 0 and n0 such that 0 <= c g(n) <= f(n) for every n >= n0.
- **Big Theta, Theta(g(n)): tight bound.** There are constants c1, c2 > 0 and n0 such that 0 <= c1 g(n) <= f(n) <= c2 g(n) for every n >= n0.
- **little-o, o(g(n)): strict upper growth bound.** f(n)/g(n) tends to 0.
- **little-omega, omega(g(n)): strict lower growth bound.** f(n)/g(n) tends to infinity.

Example: f(n) = 3n^2 + 5n + 2 is O(n^2), Omega(n^2), and Theta(n^2). Its lower-order terms and constants do not affect the dominant growth.

Common order from slower to faster growth: **1 < log n < n < n log n < n^2 < n^3 < 2^n < n!**

For a 6-mark answer, define O, Omega and Theta, give the formal inequality or plain-language meaning, and provide one example. Add little-o/little-omega if asked for all notations.

4. Recurrence equations: iteration and substitution

A recurrence describes the running time of a recursive algorithm, for example T(n) = 2T(n/2) + n.

Iteration method (repeated expansion)

Example A: T(n) = T(n/2) + c, T(1) = 1.

T(n) = T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c = T(n/2^k) + kc.

Stop when n/2^k = 1, so k = log2(n). Therefore T(n) = T(1) + c log2(n), giving **Theta(log n)**.

Example B: T(n) = T(n-1) + n, T(1) = 1.

Expand: T(n) = T(1) + 2 + 3 + … + n. Using 1 + 2 + … + n = n(n+1)/2, T(n) = Theta(n^2).

Example C: T(n) = 2T(n/2) + n, T(1) = constant.

At each level, the non-recursive work totals n. The recursion has log2(n) levels, with a constant-size leaf cost per leaf and n leaves. Thus total work is Theta(n log n).

Substitution method

1. Guess a bound based on the recurrence or expansion.
2. Assume the bound holds for smaller inputs.
3. Substitute the inductive assumption into the recurrence.
4. Choose constants so that the desired bound follows.

For T(n) = 2T(n/2) + n, guess T(n) = O(n log n). If T(n/2) <= c(n/2)log2(n/2), then T(n) <= 2[c(n/2)log2(n/2)] + n = cn(log2 n - 1) + n = cn log2 n - cn + n <= cn log2 n, for c >= 1. The matching lower bound gives Theta(n log n).

Exam tip: show each expansion and state the stopping condition. Do not write only the final complexity.

5. Divide and Conquer

Divide and Conquer breaks a problem into smaller subproblems of the same type, solves them recursively, and combines their answers.

Control abstraction

```
DivideAndConquer(P):
    if P is small enough:
        solve P directly
    divide P into smaller subproblems P1, P2, ...
    solve each subproblem recursively
    combine their results
    return the combined result
```

Examples include Merge Sort, Quick Sort and divide-and-conquer matrix multiplication.

Merge Sort

Idea: split the array into two halves, sort both halves recursively, and merge the sorted halves.

```
MERGE-SORT(A, left, right):
    if left >= right: return
    mid = floor((left + right) / 2)
    MERGE-SORT(A, left, mid)
    MERGE-SORT(A, mid + 1, right)
    MERGE(A, left, mid, right)
```

Illustration: [38, 27, 43, 3] - Split into [38, 27] and [43, 3]. - Sort the halves: [27, 38] and [3, 43]. - Merge in increasing order: **[3, 27, 38, 43]**.

Complexity: T(n) = 2T(n/2) + Theta(n) = Theta(n log n) in best, average and worst cases. Auxiliary space is Theta(n). Standard Merge Sort is stable.

Quick Sort

Idea: select a pivot, partition the elements into values smaller and larger than the pivot, then recursively sort the partitions.

```
QUICK-SORT(A, low, high):
    if low < high:
        p = PARTITION(A, low, high)
        QUICK-SORT(A, low, p - 1)
        QUICK-SORT(A, p + 1, high)
```

Example with last element as pivot: [23, 62, 27, 10, 15] - Pivot 15: partition gives [10, 15, 27, 23, 62] (the pivot is in its final position; values after it may still need sorting). - Sort the right side [27, 23, 62]. Using 62 as pivot does not change that group; a later partition places 23 before 27. - Final sorted list: **[10, 15, 23, 27, 62]**.

Complexity: best/average Theta(n log n); worst Theta(n^2), when partitions are repeatedly very unbalanced (for example, the pivot is always the smallest/largest item). Typical in-place implementations use average recursion space O(log n), worst O(n). Quick Sort is not generally stable.

Merge Sort versus Quick Sort

Feature	Merge Sort	Quick Sort
Main operation	Merge sorted halves	Partition around pivot
Best/average/worst time	Theta(n log n) for all	Theta(n log n) best/average; Theta(n^2) worst
Extra array space	Theta(n)	Usually in-place partition; recursion stack
Stability	Stable in standard implementation	Not generally stable
Common issue	Additional memory	Bad pivot can cause unbalanced partitions

Matrix multiplication using divide and conquer

Split each n x n matrix into four n/2 x n/2 blocks. For A = [[A11,A12],[A21,A22]] and B = [[B11,B12],[B21,B22]], compute: - C11 = A11B11 + A12B21 - C12 = A11B12 + A12B22 - C21 = A21B11 + A22B21 - C22 = A21B12 + A22B22

The standard block method performs 8 recursive matrix multiplications and Theta(n^2) additions: **T(n) = 8T(n/2) + Theta(n^2) = Theta(n^3)**.

Strassen's method reduces the number of recursive multiplications from 8 to 7, yielding T(n) = 7T(n/2) + Theta(n^2) = Theta(n^(log2 7)) approximately **Theta(n^2.807)**. It uses more additions/subtractions to reduce multiplications.

MODULE 2 - Greedy Strategy and Dynamic Programming

6. Greedy strategy

A greedy algorithm builds a solution one step at a time, choosing the locally best available option according to a rule. It is optimal only for problems that satisfy the required greedy-choice property and structure; a locally best decision does not guarantee an optimal answer for every problem.

Greedy control abstraction

```
GREEDY(C):
    solution = empty
    while solution is not complete and candidates remain:
        x = select the best candidate by the greedy rule
        remove x from candidates
        if adding x keeps the solution feasible:
            add x to solution
    return solution
```

Fractional Knapsack

Each item has profit p_i and weight w_i; capacity is W. Sort by decreasing **profit density p_i/w_i**. Take an item completely while it fits; when the next item does not fit, take the required fraction and stop.

Example: capacity W = 30. Items A, B, C, D have values 50, 140, 60, 60 and weights 5, 20, 10, 12. - Densities: A = 10, B = 7, C = 6, D = 5. - Take A (weight 5, profit 50); B (weight 20, profit 140); remaining capacity = 5. - Take 5/10 of C for profit 30. - Total weight = 30; total profit = **220**.

Time complexity: O(n log n) for sorting the ratios. The greedy ratio method is optimal for Fractional Knapsack. It does not generally solve 0/1 Knapsack optimally, because 0/1 items cannot be split.

Job scheduling with deadlines

Assume each job takes one unit of time and has a deadline and profit; a job earns its profit only if completed by its deadline. 1. Sort jobs in decreasing order of profit. 2. For each job, place it in the latest free time slot at or before its deadline. 3. If no such slot exists, reject that job. 4. The scheduled jobs maximize profit under the usual unit-time model.

A common 6-mark answer should state the assumptions, sorting rule, slot-placement rule, and the final schedule/profit for any supplied table.

7. Minimum Spanning Tree (MST)

A spanning tree of a connected, undirected, weighted graph connects every vertex, has no cycle, and contains V - 1 edges. A **minimum spanning tree** is a spanning tree with minimum total edge weight.

Kruskal's algorithm

1. Sort all edges in nondecreasing order of weight.
2. Start with an empty MST.
3. Examine edges in order; add an edge if it connects two different components (does not create a cycle).
4. Stop after V - 1 edges.
5. Use Disjoint Set Union/Union-Find to efficiently check and merge components.

Complexity: O(E log E), dominated by sorting edges.

Prim's algorithm

1. Start from any vertex; put it in the growing tree.
2. Repeatedly select the minimum-weight edge connecting a vertex inside the tree to a vertex outside it.
3. Add that edge and new vertex; do not create a cycle.
4. Stop when all vertices are included.

Complexity: O(V^2) with an adjacency matrix/simple implementation; O(E log V) with an adjacency list and priority queue.

Prim versus Kruskal

- Prim grows one connected tree from a starting vertex.
- Kruskal grows a forest and joins components using the next-lowest safe edge.
- Prim chooses the cheapest edge crossing the current tree cut; Kruskal chooses the next-lowest edge that does not form a cycle.
- Both produce an MST for a connected, undirected, weighted graph.

Worked-answer checklist for graph questions: list chosen edges in order, show why each is accepted/rejected, ensure exactly V - 1 edges, and add their weights for the total MST cost.

8. Dynamic Programming (DP)

Dynamic Programming solves a problem by combining solutions to overlapping subproblems and storing their results so that the same subproblem is not repeatedly solved.

Control abstraction

```
DYNAMIC-PROGRAMMING:
    identify the state/subproblems
    set base cases
    derive a recurrence from smaller states
    compute states in dependency order and store results
    return the value/state representing the required solution
```

Principle of optimal substructure: an optimal solution to a problem can be built from optimal solutions to its relevant subproblems. DP is especially useful when the subproblems overlap.

All-pairs shortest paths - Floyd-Warshall

Let D[i][j] be the shortest known distance from i to j. Initialize D with edge weights, D[i][i] = 0, and infinity where there is no edge. Then allow each vertex k to be used as an intermediate vertex:

```
FLOYD-WARSHALL(D):
    for k = 1 to V:
        for i = 1 to V:
            for j = 1 to V:
                D[i][j] = min(D[i][j], D[i][k] + D[k][j])
    return D
```

Recurrence: **D_k[i][j] = min(D_(k-1)[i][j], D_(k-1)[i][k] + D_(k-1)[k][j])**.

Time complexity is O(V^3); space is O(V^2). It permits negative edge weights but not negative-weight cycles for meaningful shortest paths. A negative value on D[i][i] after the algorithm indicates a negative cycle reachable in the matrix's graph.

Travelling Salesperson Problem (TSP) using DP

Given cities and travel costs, find a minimum-cost tour that starts at a city, visits every other city exactly once, and returns to the start.

For a fixed start city 1, let DP[S][j] be the minimum cost of a path starting at 1, visiting the cities in set S, and ending at j. - Base case: DP[{1,j}][j] = cost(1,j). - Transition: DP[S][j] = min over i in S, i != j of (DP[S - {j}][i] + cost(i,j)). - Final answer: min over j != 1 of (DP[all cities][j] + cost(j,1)).

The standard Held-Karp algorithm takes **O(n^2 2^n)** time and O(n 2^n) space. In an exam numerical, write the subset/state definition, fill the relevant DP values, and add the return-to-start cost. TSP is computationally expensive as n grows.

Bellman-Ford - single-source shortest paths

Bellman-Ford computes shortest distances from source s, even with negative edge weights, and detects reachable negative-weight cycles. 1. Set dist[s] = 0; every other distance = infinity. 2. Repeat V - 1 times: for every directed edge (u,v) of weight w, if dist[u] + w < dist[v], update dist[v]. This is called relaxing the edge. 3. Scan all edges one more time. If any distance can still be reduced, a reachable negative-weight cycle exists.

```
BELLMAN-FORD(G, w, s):
    for each vertex v: dist[v] = infinity
    dist[s] = 0
    repeat V - 1 times:
        for each edge (u, v):
            if dist[u] != infinity and dist[u] + w(u,v) < dist[v]:
                dist[v] = dist[u] + w(u,v)
    for each edge (u, v):
        if dist[u] != infinity and dist[u] + w(u,v) < dist[v]:
            report reachable negative-weight cycle
    return dist
```

Time complexity O(VE); space O(V). Dijkstra's algorithm should not be used with arbitrary negative edge weights; Bellman-Ford can handle them, provided no reachable negative cycle invalidates shortest paths.

9. Fast comparison: Greedy versus DP

Greedy	Dynamic Programming
Makes a locally best choice at each step	Evaluates combinations of subproblem solutions
Decisions are typically not reconsidered	Stores and reuses subproblem results
Can be faster and simpler	Often uses more time/memory
Needs a valid greedy-choice property for optimality	Uses recurrence plus optimal substructure; overlapping subproblems are common
Examples: fractional knapsack, Prim, Kruskal, job scheduling	Examples: Floyd-Warshall, TSP DP, Bellman-Ford-style relaxation is a shortest-path method (not typically described as classic DP in every text)

MODULE 5 - Approximation Algorithms (only the specified topics)

10. Approximation algorithms and approximation ratio

An approximation algorithm is a polynomial-time algorithm for an optimization problem that returns a feasible solution with a provable bound on how far its value can be from the optimum. It does **not** necessarily return the exact optimal solution.

For a minimization problem, a common ratio guarantee is: **ALG / OPT <= rho**, where ALG is the cost of the algorithm's solution, OPT is the optimal cost, and rho >= 1. For a 2-approximation, ALG <= 2 OPT.

For maximization problems, ratio conventions are sometimes written in the reciprocal form; state the convention being used. For Vertex Cover (a minimization problem), use |C| <= 2|C*|.

11. 2-approximation algorithm for Vertex Cover

A vertex cover C is a set of vertices such that every edge has at least one endpoint in C.

Algorithm:

```
APPROX-VERTEX-COVER(G):
    C = empty set
    E' = all edges of G
    while E' is not empty:
        choose any edge (u, v) from E'
        add u and v to C
        remove from E' every edge incident to u or v
    return C
```

Why it is a vertex cover: the loop stops only when every edge has been removed. Each removed edge is incident to u or v that was added to C, so every original edge has an endpoint in C.

Why its approximation ratio is at most 2: the selected edges are pairwise disjoint, because after selecting (u,v), every edge incident to u or v is removed. These selected edges form a matching M. Any vertex cover must select at least one endpoint of every edge in M, so OPT >= |M|. The algorithm selects both endpoints of each selected edge, so |C| = 2|M| <= 2OPT. Therefore it is a **2-approximation**.

Exam answer format: definition of vertex cover; pseudocode/steps; brief correctness argument; matching lower-bound argument; conclude |C| <= 2OPT.

12. Vertex Cover using Linear Programming and LP Rounding

Integer programming formulation

For each vertex v, define x_v = 1 if v is selected, otherwise x_v = 0. Minimize the number of selected vertices:

Minimize: **sum over v in V of x_v**

Subject to: **x_u + x_v >= 1** for every edge (u,v) in E, and x_v in {0,1} for every vertex v.

The constraint ensures every edge is covered by at least one endpoint.

LP relaxation

Replace the integer condition x_v in {0,1} with **0 <= x_v <= 1**. This allows fractional values and forms a linear program. Because the feasible region is enlarged, the optimal relaxed objective LP* is a lower bound on the integral optimum OPT: LP* <= OPT.

LP rounding algorithm

1. Solve the relaxed LP and obtain values x_v.
2. Select every vertex v with x_v >= 1/2.
3. Return the selected vertices as the cover C.

Why rounding returns a valid cover

For every edge (u,v), x_u + x_v >= 1. Therefore at least one of x_u or x_v must be >= 1/2. The rounding rule selects that endpoint, so every edge is covered.

Approximation-ratio proof

Every selected vertex has x_v >= 1/2, so 1 <= 2x_v. Summing over selected vertices:

|C| <= 2 * sum over all vertices of x_v = 2LP* <= 2OPT.

Thus LP rounding gives a **2-approximation** for unweighted Vertex Cover. For the standard weighted formulation with nonnegative costs, the analogous cost bound is cost(C) <= 2LP* <= 2OPT.

Compare the two 2-approximation methods

- **Edge-picking method:** repeatedly choose an uncovered edge and add both endpoints; proof uses the matching of selected edges.

- **LP rounding method:** solve the LP relaxation and select all vertices with x_v >= 1/2; proof uses the LP lower bound and threshold property.
- Both guarantee a solution no more than twice optimum for Vertex Cover, but their procedures and proofs differ.

LAST-MINUTE QUESTION BANK

Likely 3-mark preparation (practice concise definitions)

1. Define time complexity and space complexity. Distinguish best, worst and average cases.
2. Define Big O, Big Omega and Big Theta with examples.
3. Write the control abstraction of Divide and Conquer.
4. Write the control abstraction of Greedy Strategy.
5. State the principle of optimal substructure.
6. What is a minimum spanning tree? Define approximation ratio.
7. Why does Quick Sort have a worst-case time of O(n^2)?
8. Solve T(n) = T(n/2) + c by iteration.
9. Define Fractional Knapsack and explain why profit/weight ratio is used.
10. What is LP rounding for Vertex Cover?

Likely 6-mark preparation - Module 1

1. Explain O, Omega and Theta notations with definitions and examples.
2. Write Merge Sort, trace a given array, and analyze its time and space complexity.
3. Write Quick Sort, trace a given array, and explain best/average/worst cases.
4. Solve T(n) = 2T(n/2) + n and T(n) = T(n-1) + n using iteration/substitution.
5. Explain divide-and-conquer matrix multiplication and derive O(n^3); compare with Strassen's method if asked.

Likely 6-mark preparation - Module 2

1. Write Kruskal's algorithm and find an MST from a weighted graph.
2. Write Prim's algorithm and calculate the MST cost from a given graph.
3. Solve a Fractional Knapsack numerical using profit/weight ratios.
4. Explain Floyd-Warshall and work through the distance matrix.
5. Explain the DP solution for TSP and show the recurrence/state definition.
6. Explain Bellman-Ford, edge relaxation, complexity, and negative-cycle detection.
7. Explain greedy job scheduling with deadlines and solve a small schedule.

Likely 6-mark preparation - Module 5 (allowed portion only)

1. Define approximation ratio and explain a 2-approximation algorithm for Vertex Cover with proof.
2. Formulate Vertex Cover as an LP, relax the integer constraints and explain the rounding algorithm.
3. Prove that LP rounding is a 2-approximation.
4. Compare edge-picking Vertex Cover approximation and LP rounding.

Night-before revision order

1. **First:** asymptotic notations; Merge Sort; Quick Sort; recurrence iteration/substitution.
2. **Second:** Prim and Kruskal; Fractional Knapsack; Floyd-Warshall; TSP; Bellman-Ford.
3. **Third:** Vertex Cover 2-approximation and its proof; LP formulation and LP rounding proof.
4. **Finally:** Greedy/Divide-and-Conquer/DP control abstractions, complexity definitions, job scheduling.

Accuracy reminders

- MST contains exactly V - 1 edges for a connected graph.
- Fractional Knapsack sorts by profit/weight, not by profit alone or weight alone.
- Standard Merge Sort is Theta(n log n) in all cases; standard Quick Sort can be Theta(n^2) in the worst case.
- In Bellman-Ford, perform V - 1 relaxation passes, then one extra cycle-detection pass.
- In Floyd-Warshall, loop over k outside the i and j loops in the standard recurrence.
- A 2-approximation for minimization guarantees ALG <= 2OPT; it does not claim the algorithm is exactly optimal.
- LP rounding for Vertex Cover selects x_v >= 1/2 and has the bound |C| <= 2LP* <= 2OPT.

WORKED NUMERICALS TO PRACTISE

Use these as exam-writing practice: show the rule, the intermediate steps, and the final result. Some examples are taken from the uploaded PYQs; others are small worked examples to practise the exact method.

13. Recurrence equations: quick solved set

A. T(n) = T(n/2) + c, T(1) = 1

Repeated expansion:

T(n) = T(n/2) + c
= T(n/4) + 2c
= T(n/8) + 3c
= T(n/2^k) + kc.

Stop at n/2^k = 1, giving k = log2 n. Therefore T(n) = 1 + c log2 n = **Theta(log n)**.

B. T(n) = T(n - 1) + n, T(1) = 1

Expand until the base case:

T(n) = T(1) + 2 + 3 + … + n = 1 + [n(n + 1)/2 - 1] = n(n + 1)/2.

Therefore **T(n) = Theta(n^2)**.

C. T(n) = 2T(n/2) + n

At each level, the total non-recursive work is n. The recursion tree has log2 n levels, so total work is n log2 n plus leaf-level work of order n. Therefore **T(n) = Theta(n log n)**.

D. T(n) = 2T(n/2) + 1

The recursion tree has n leaves. The sum of constant work at internal nodes is 1 + 2 + 4 + … + n/2 = n - 1, and leaf work is also proportional to n. Therefore **T(n) = Theta(n)**.

Remember: state the recurrence, expand or draw the tree, identify the stopping level/base case, simplify the sum, and write the final asymptotic bound.

14. Fractional Knapsack PYQ numerical

A December/January 2023-24 paper gives capacity W = 15 and the following items:

Item	1	2	3	4	5	6	7
Profit p	15	4	9	7	8	10	5
Weight w	5	2	3	4	1	3	1
p/w	3	2	3	1.75	8	3.33	5

Sort by decreasing p/w: item 5, item 7, item 6, items 1 and 3 (tie), then item 2, then item 4.

Take items 5, 7, 6, 1, 3 and 2 fully. Their total weight is 1 + 1 + 3 + 5 + 3 + 2 = 15. Their total profit is 8 + 5 + 10 + 15 + 9 + 4 = **51**. Item 4 is not selected. Capacity is exactly filled and the maximum profit is **51**.

Exam trap: choosing by profit alone is wrong. Fractional Knapsack uses profit divided by weight.

15. A small MST example for Prim and Kruskal

Consider an undirected graph with these edges:

Edge	Weight
A-B	1
A-C	4
B-C	2
B-D	5
C-D	3

Kruskal

1. Sort: AB(1), BC(2), CD(3), AC(4), BD(5).
2. Select AB(1).
3. Select BC(2); it does not create a cycle.
4. Select CD(3); it does not create a cycle and now all four vertices are connected.
5. Stop after V - 1 = 3 edges.

MST = {AB, BC, CD}; total cost = 1 + 2 + 3 = **6**.

Prim (start at A)

1. From A, choose AB(1).
2. Among edges connecting the current tree {A,B} to an outside vertex, choose BC(2).
3. From {A,B,C}, choose CD(3).

The total is again **6**. In an exam, follow the weights in the supplied graph rather than memorising these sample edges.

16. Floyd-Warshall: worked 3-vertex matrix

Initial cost matrix (infinity means no direct edge):

```
      1   2   3
1     0   4  11
2    inf  0   2
3     3  inf  0
```

Use D[i][j] = min(D[i][j], D[i][k] + D[k][j]), with k as the outermost loop.

- Allow vertex 1 as an intermediate: path 3 -> 1 -> 2 has cost 3 + 4 = 7, so D[3][2] becomes 7.
- Allow vertex 2: path 1 -> 2 -> 3 has cost 4 + 2 = 6, so D[1][3] becomes 6.
- Allow vertex 3: path 2 -> 3 -> 1 has cost 2 + 3 = 5, so D[2][1] becomes 5.

Final shortest-distance matrix:

```
      1  2  3
1     0  4  6
2     5  0  2
3     3  7  0
```

Complexity: **O(V^3) time and O(V^2) space**. In a full question, show each updated matrix or clearly list changed cells at each k.

17. Bellman-Ford: worked relaxation example

Directed edges: s->a (4), s->b (5), a->b (-2), b->c (3), a->c (4). Source is s.

Initialize: d(s) = 0; d(a) = d(b) = d(c) = infinity.

Relax edges: - s->a: d(a) = 4. - s->b: d(b) = 5. - a->b: d(b) = min(5, 4 - 2) = 2. - b->c: d(c) = min(infinity, 2 + 3) = 5. - a->c: d(c) = min(5, 4 + 4) = 5 (no change).

Final distances: **d(s)=0, d(a)=4, d(b)=2, d(c)=5**. Since there are V = 4 vertices, do V - 1 = 3 full relaxation passes in the general algorithm, then one additional scan to detect a reachable negative cycle.

18. TSP: small route example and answer structure

For four cities A, B, C and D, suppose the symmetric costs are:

From/to	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Two minimum tours are A -> B -> D -> C -> A and A -> C -> D -> B -> A. Each costs 10 + 25 + 30 + 15 = **80**. This small example helps check the interpretation of the tour; a DP solution should still define its state, give the transition, and show the state values rather than merely listing routes.

State form with A as the start: DP[S][j] is the least cost from A, visiting the subset S, and ending at j. Transition: DP[S][j] = min over i in S excluding j of {DP[S minus {j}][i] + cost(i,j)}. Add the final edge back to A to obtain the tour cost.

19. Vertex Cover: example, LP formulation, and rounding

Consider a triangle graph with vertices A, B, C and edges AB, BC, CA.

Edge-picking 2-approximation

1. Choose edge AB and add both A and B to C.
2. Remove every edge incident to A or B. All three edges are removed.
3. Return C = {A, B}. It covers every edge. Here |C| = 2 and OPT = 2.

For the general proof, if the algorithm selects m pairwise vertex-disjoint edges, it returns 2m endpoints. Any vertex cover must select at least one endpoint from each of those m edges, so OPT >= m. Thus |C| = 2m <= 2OPT.

LP relaxation and rounding

Integer program for unweighted Vertex Cover:

```
minimize    sum_v x_v
subject to  x_u + x_v >= 1     for every edge (u,v)
            x_v in {0,1}       for every vertex v
```

Relax x_v in {0,1} to 0 <= x_v <= 1. Solve the LP, then choose every vertex with x_v >= 1/2. For each edge, x_u + x_v >= 1 means at least one endpoint has value >= 1/2, so that endpoint is selected and the edge is covered.

Let LP* be the relaxed optimum. Every selected vertex has x_v >= 1/2, so |C| <= 2 sum_v x_v = 2LP* <= 2OPT. Therefore this is a **2-approximation**. If weights c_v are provided, minimise sum c_v x_v and apply the same threshold; the cost proof is cost(C) <= 2LP* <= 2OPT, assuming nonnegative vertex costs.

FINAL 30-MINUTE CHECKLIST

- ☐ Define time complexity, space complexity, best/worst/average case.
- ☐ Write formal Big O, Omega and Theta definitions.
- ☐ Solve T(n)=T(n/2)+c and T(n)=T(n-1)+n by iteration.
- ☐ Write Merge Sort pseudocode, trace an array, state Theta(n log n).
- ☐ Write Quick Sort partition idea and explain worst-case Theta(n^2).
- ☐ State standard divide-and-conquer matrix multiplication recurrence and Strassen recurrence.

- ☐ Write Greedy control abstraction and fractional knapsack ratios.
- ☐ Practise Prim and Kruskal by listing accepted edges and total cost.
- ☐ Write job sequencing steps: descending profit and latest available slot before deadline.
- ☐ Write Floyd-Warshall recurrence with k outermost.
- ☐ Write TSP state, transition and final return edge.
- ☐ Write Bellman-Ford V-1 passes plus an extra negative-cycle check.
- ☐ Memorise Vertex Cover 2-approx matching proof.
- ☐ Memorise LP constraints and the x_v >= 1/2 rounding proof.

Scope reminder: Do not spend tonight revising backtracking, 8-puzzle, lower-bound theory, randomized quicksort, or Schwartz-Zippel for this stated exam scope.